

Generative Adversarial Networks

Hyoungh Joon, Kim

June 3, 2026

Contents

1	Introduction	2
2	Main Idea	2
2.1	Generator and Discriminator	2
2.2	Training GAN	2
2.2.1	Loss Function for GAN	2
2.2.2	Training Generator and Discriminator	2
3	Extensions of GAN	3
3.1	WGAN	3
3.1.1	Introduction	3
3.1.2	Analysis of GAN Loss Function	3
3.1.3	Vanishing Gradients	4
3.1.4	EM Distance	4
3.1.5	WGAN	4

1 Introduction

A **generative adversarial network** (GAN) is an unsupervised network, which aims to generate new samples that are indistinguishable from original data. GAN is just an architecture for creating new samples, so it cannot evaluate probability if a new sample belongs to the original distribution.

2 Main Idea

2.1 Generator and Discriminator

GAN uses two distinct networks; the **discriminator** and **generator**. The discriminator network classifies a given data; it classifies if the given data is created by generator or not. The generator network's objective is to create samples that can deceive a discriminator. The idea is simple, but GAN itself is hard to train. We'll discuss more about this in section 3.

The generator creates new samples by first picking a latent variable $\mathbf{z}$ from some base distribution and passing this variable through a network $\mathbf{g}(\mathbf{z}, \boldsymbol{\theta})$ with parameter $\boldsymbol{\theta}$. The goal is to find parameter $\boldsymbol{\theta}$ that creates sample indistinguishable from original dataset. This indistinguishableness, is defined to be statistically indistinguishable from the true data.

The discriminator network $\mathbf{f}(\mathbf{x}, \boldsymbol{\phi})$ with parameter $\boldsymbol{\phi}$ discriminates whether the given data $\mathbf{x}$ is created by generator or not.

2.2 Training GAN

2.2.1 Loss Function for GAN

The GAN uses binary cross entropy loss for training discriminator. Generator is trained to increase the loss, when discriminator parameter is fixed.

Definition 2.2.1 (Loss Function for GAN)

Let $\mathbf{g}(\mathbf{z}, \boldsymbol{\theta})$ and $\mathbf{f}(\mathbf{x}, \boldsymbol{\phi})$ be generator and discriminator, respectively, where $\mathbf{z}_j$ is a latent variable and $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$ are parameters. Let $\mathbf{y}_j$ be a sample created by the generator with latent variable $\mathbf{z}_j$, and let $\mathbf{x}_i$ be the original data. Then, the loss for GAN is defined as following:

$$\sum_j -\log(1 - \sigma(\mathbf{f}(\mathbf{y}_j, \boldsymbol{\phi}))) - \sum_i -\log(\sigma(\mathbf{f}(\mathbf{x}_i, \boldsymbol{\phi})))$$

which can be written as

$$\sum_j -\log(1 - \sigma(\mathbf{f}(\mathbf{g}(\mathbf{z}_j, \boldsymbol{\theta}), \boldsymbol{\phi}))) - \sum_i -\log(\sigma(\mathbf{f}(\mathbf{x}_i, \boldsymbol{\phi})))$$

2.2.2 Training Generator and Discriminator

Given the loss function above, the discriminator first tries to minimize the loss function, by adjusting $\boldsymbol{\phi}$. Then, the generator tries to maximize the loss with $\boldsymbol{\phi}$ fixed, by adjusting $\boldsymbol{\theta}$. Technically, the solution is *Nash Equilibrium*.

Definition 2.2.2 (Training Discriminator)

The loss function for the discriminator is

$$\mathcal{L}_{discriminator}(\phi) = \sum_j -\log(1 - \sigma(\mathbf{f}(\mathbf{g}(\mathbf{z}_j, \theta), \phi))) + \sum_i -\log(\sigma(\mathbf{f}(\mathbf{x}_i, \phi)))$$

using the notation of Definition 2.2.1.

Definition 2.2.3 (Training Generator)

The loss function for the generator is

$$\mathcal{L}_{generator}(\theta) = \sum_j \log(1 - \sigma(\mathbf{f}(\mathbf{g}(\mathbf{z}_j, \theta), \phi)))$$

using the notation of Definition 2.2.1. The second function is dropped since it has no dependence on θ , and the leftover is multiplied by -1.

We draw a batch of latent variables $\mathbf{z}_j$, and create samples. Then we choose a batch of real examples $\mathbf{x}_i$. Then we perform gradient descents on each loss function.

3 Extensions of GAN

3.1 WGAN

3.1.1 Introduction

The original GAN architecture is difficult to train. The training itself is unstable, and even though it is trained well, it may suffer from **mode dropping** and **mode collapse**. If the generator successfully deceives discriminator but creates only a subset of whole data, this is called mode dropping. If this phenomenon gets extreme, the generator creates only one sample, ignoring the latent variable. This is called mode collapse.

3.1.2 Analysis of GAN Loss Function

To figure out why the mode dropping and collapse happen, let's examine the loss function of GAN more precisely.

Theorem 3.1.1 (Analysis of GAN Loss Function)

Let p be the original distribution of the data, and let q be the the distribution over the generated samples. Then, the loss of GAN function can be interpreted as the **Jensen-Shannon divergence**, ignoring the additive and multiplicative constants;

$$\mathcal{L}_{discriminator}(\phi) = - \int q(\mathbf{x}) \log \frac{q(\mathbf{x})}{q(\mathbf{x}) + p(\mathbf{x})} d\mathbf{x} - \int p(\mathbf{x}) \log \frac{p(\mathbf{x})}{q(\mathbf{x}) + p(\mathbf{x})} d\mathbf{x}$$

where Jensen-Shannon divergence is defined as:

$$D_{JS}(q||p) = \frac{1}{2} \int q(x) \log \frac{2q(\mathbf{x})}{q(\mathbf{x}) + p(\mathbf{x})} d\mathbf{x} + \frac{1}{2} \int p(x) \log \frac{2p(\mathbf{x})}{q(\mathbf{x}) + p(\mathbf{x})} d\mathbf{x}$$

Whenever the sample density $q(x)$ is high and the mixture $q(x) + p(x)$ has high probability, the first term of JS divergence will be small. That is, the first term penalizes regions where there generated samples are plenty but real samples are not; it enforces *quality*. Vice versa, the second term penalizes regions with plenty of real samples and sparse generated samples; it enforces *coverage*. Since the second term does not depend on generator, generator does not care about the coverage of the samples being created. This can be a reason of mode dropping and collapse.

3.1.3 Vanishing Gradients

When the discriminator is optimal, it maximizes the JS-divergence between two distributions. If the probability distributions are completely disjoint, this distance is maximal, which will be a constant of $\log 2$. So any small change to the generator won't decrease the loss. The problem is that the distributions of samples and original dataset may really be disjoint, which makes training impossible.

3.1.4 EM Distance

To solve the problem, we use different distance metric called **EM distance** or **Wasserstein distance**. EM distance is defined as follows:

Definition 3.1.1 (EM Distance)

Let p and q be probability distribution. The **EM distance** $W(p, q)$ between p and q is defined by the equation

$$W(p, q) = \inf_{\gamma \in \Pi(p, q)} \mathbb{E}_{(x, y) \sim \gamma} [\|x - y\|]$$

where $\Pi(p, q)$ is sets of all possible joint distribution over p and q .

We can think of γ as a transport plan. The EM distance, as we can see from its name, can be understood as the minimum transport cost used to transport p to q . This can be understood better when the distributions are discrete.

Using this equation directly is impossible since finding the infimum in high-dimensional space is tremendously costly. So, we use the duality property of the optimization theory. Then, the equation can be written as

$$W(p, q) = \sup_{\|f\|_L \leq 1} (\mathbb{E}_{x \sim p}[f(x)] - \mathbb{E}_{y \sim q}[f(y)]),$$

where $\|f\|_L \leq 1$ denotes the Lipschitz condition, $\|\nabla_{\mathbf{x}} f(\mathbf{x}, \phi)\| \leq 1$. Now the objective is to find the function f , called **critic**, which is usually trained with neural network.

3.1.5 WGAN

We now define Wasserstein GAN loss function and use it to train GAN.

Definition 3.1.2 (Wasserstein GAN Loss Function)

Let $\mathbf{z}_j$ be a latent variable and let $\mathbf{x}_i$ be a given real samples. The **Wasserstein GAN loss function** is given as following:

$$\mathcal{L} = \sum_j f(\mathbf{g}(\mathbf{z}_j, \boldsymbol{\theta}), \phi) - \sum_i f(\mathbf{x}_i, \phi), \text{ with } \|\nabla_{\mathbf{x}} f(\mathbf{x}, \phi)\| \leq 1$$

where $\mathbf{g}$ is a generator network, and f is a critic network.

The expectation terms are approximated with sums about samples, and the condition $\|\nabla_{\mathbf{x}} f(\mathbf{x}, \phi)\| \leq 1$ is called **Lipschitz condition**.

To achieve this, we use one of the following methods:

1. Clip the gradient; This is the what the original WGAN uses.
2. Add a regularization term, penalizing when the gradient norm is bigger than one; This is called WGAN-GP, where GP stands for *gradient penalty*.

The *critic* has same objective as the discriminator of original GAN. However, the critic does not produce probability. Instead, it calculates a score that measures the sample's proximity to the real data distribution.